

Skin Lesion Classification with ISIC 2019 and ResNet50

A code-aware explanation of the supplied Jupyter notebook: what each part does, why it was chosen, and how the full training and evaluation pipeline fits together.

25,331	labelled images
0.9153	test ROC-AUC
0.8421	test accuracy
2 stages	transfer learning

Scope of this report

This report is based on the saved code, comments, and cell outputs in **notebook.ipynb**. It describes the experiment that was run; it does not claim clinical validity or independently reproduce the training.

Prepared from the supplied notebook

Dataset: ISIC 2019 Training Ground Truth and training images | Runtime recorded by notebook: NVIDIA GeForce RTX 4050 Laptop GPU (CUDA)

Executive summary

The notebook builds a binary classifier for dermoscopic skin-lesion images. It treats melanoma (MEL), basal cell carcinoma (BCC), and squamous cell carcinoma (SCC) as the positive class, called **malignant**. All remaining categories in the CSV are treated as the negative class, called **benign**. The notebook uses a pre-trained ResNet50 image model, first training only a new classifier head and then fine-tuning the deeper ResNet layers.

The final reported test result is promising for this experimental setup: ROC-AUC 0.9153, accuracy 0.8421, malignant recall 0.7766, and benign specificity 0.8750 on 3,800 test images. In practical terms, the model correctly identified 987 of 1,271 positive lesions and 2,213 of 2,529 negative lesions at a 0.50 probability threshold.

End-to-end workflow

Step	Notebook action	Why it matters
1. Define task	Read CSV; collapse MEL/BCC/SCC into label 1	Turns multi-column diagnosis labels into a single binary prediction target.
2. Validate files	Build image paths and confirm 25,331 files exist	Prevents training failures or silent dataset mismatch.
3. Split data	70% train, 15% validation, 15% test, stratified	Keeps class prevalence alike across splits and reserves unseen test data.
4. Prepare images	Resize, augment train images, normalize all images	Creates model-ready tensors while making training more robust.
5. Balance training	WeightedRandomSampler oversamples positives	Makes malignant images appear more often in training batches.
6. Train model	ResNet50 transfer learning in two stages	Uses prior ImageNet features before adapting them to skin lesions.
7. Evaluate	Loss/AUC during training; classification metrics on test set	Measures ranking ability and threshold-based diagnostic behavior.

1. Software stack and runtime choices

The import cells bring together data handling, image processing, neural-network training, and evaluation. Each package has a specific role in the pipeline.

Package / tool	Used for	Why useful here
PyTorch (torch, torch.nn, torch.optim)	Tensors, model layers, GPU execution, loss and optimizer	The central deep-learning framework. It lets the model train on CUDA and exposes AMP, samplers, checkpoints, and ResNet parameters.
torchvision	Pre-trained ResNet50 and image transforms	Supplies ImageNet weights and standard image augmentation / normalization tools compatible with PyTorch.
pandas	CSV loading and label tables	Makes it easy to select columns, create the binary label, inspect distributions, and split tabular metadata.
NumPy	Numeric arrays at evaluation	Supports conversion of probabilities and labels to arrays for sklearn metrics.
PIL / Pillow	Image.open(...).convert('RGB')	Reads JPEG images and guarantees three channels, which ResNet50 expects.
scikit-learn	train_test_split and metrics	Provides stratified splits, ROC-AUC, accuracy, precision, recall, F1, and confusion matrices.
tqdm	Progress bars	Shows batch-level progress and loss during lengthy training and testing.
matplotlib	Learning curves and confusion-matrix plots	Makes convergence and error patterns visible rather than leaving only numeric output.
OpenCV	Installed but not used later	The notebook installs opencv-python, but no cv2 operation appears in the saved cells. It has no active role in this version.

Hardware and speed settings

- **CUDA device selection:** the code chooses cuda when available, otherwise CPU. The recorded run used an NVIDIA GeForce RTX 4050 Laptop GPU.
- **TF32 and cuDNN benchmark:** CUDA settings allow faster matrix operations and choose fast convolution algorithms. They are performance choices, not changes to the learning objective.
- **AMP:** automatic mixed precision performs suitable GPU operations in float16 while GradScaler protects gradients from underflow. This reduces memory use and often speeds training.
- **channels_last, pin_memory, non_blocking transfer:** these optimize how image data moves through CPU memory to the GPU. The notebook benchmarks about 0.55 seconds per training batch in one speed test.
- **torch.compile:** prepared as an optional optimization but explicitly disabled (False), so it was not part of this recorded training run.

2. Dataset preparation and label design

The source file has 25,331 rows and diagnosis indicator columns: MEL, NV, BCC, AK, BKL, DF, VASC, SCC, and UNK. The notebook intentionally retains only image, MEL, BCC, and SCC before constructing a binary label.

Binary target definition

The label is created with logical OR: `label = 1 if MEL == 1 or BCC == 1 or SCC == 1; otherwise 0`. This is the key modelling decision. It frames the model as a broad screen for three clinically important cancer categories rather than a classifier of all individual ISIC diagnoses.

Why this is useful

A single output neuron is simpler to train and evaluate than a nine-class model, and the positive class has 8,473 images. The trade-off is that the label *benign* is shorthand: it includes all non-positive categories remaining in the CSV, not a separately verified clinical benign-only cohort.

Class after relabelling	Count	Share	Meaning in this notebook
0 - Benign	16,858	66.55%	Any row not labelled MEL, BCC, or SCC.
1 - Malignant	8,473	33.45%	MEL, BCC, or SCC.
Total	25,331	100.00%	All CSV records; file verification reports zero missing JPEGs.

File verification

A `pathlib Path` is made for each image ID, ending in `.jpg`. The notebook checks the first file and counts every path. The output says 25,331 images found and 0 missing. This is a useful integrity check because model code can otherwise fail only later when a data-loader worker reaches a bad row.

Data split

Two calls to `train_test_split` create 70% training and then divide the temporary 30% equally into validation and test. The `stratify` argument preserves the 66.55% / 33.45% class balance in every split. `random_state=42` makes these particular splits reproducible when the same input order is used.

Split	Images	Benign	Malignant	Malignant share
Training	17,731	11,800	5,931	33.45%
Validation	3,800	2,529	1,271	33.45%
Test	3,800	2,529	1,271	33.45%

3. Image pipeline and class imbalance

The custom ISICDataset class is the bridge between a pandas table and the neural network. For an index, it obtains the stored path and label, opens the JPEG, converts it to RGB, applies the selected transform, converts the label to a float32 tensor, and returns (image, label). float32 is appropriate because BCEWithLogitsLoss expects a continuous target tensor with 0.0 or 1.0 values.

Transforms

Operation	Applied to	Purpose and reason
Resize to 224 x 224	Train, validation, test	Matches the standard ResNet50 input size; also makes batch tensors uniform: [64, 3, 224, 224].
Random horizontal / vertical flip	Train only	Models orientation changes that should not alter lesion class. Adds variation without adding new images.
Random rotation up to 20 degrees	Train only	Reduces reliance on a lesion's camera orientation.
ColorJitter: brightness, contrast = 0.2	Train only	Encourages tolerance to modest illumination and contrast differences across photos.
ToTensor	All splits	Converts an image into PyTorch channel-first numeric values.
ImageNet normalization	All splits	Uses mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225], matching the statistics assumed by the pre-trained ResNet50 weights.

Why validation and test have no augmentation

Evaluation uses only resize, tensor conversion, and normalization. That provides stable and comparable measurements. Random transformations during validation or testing would cause scores to depend on random image changes rather than only on the trained model.

Weighted sampling

The training set is moderately imbalanced: 11,800 negative versus 5,931 positive images. The code gives each class the inverse of its count: 1/11,800 for benign and 1/5,931 for malignant. WeightedRandomSampler then draws 17,731 samples with replacement per epoch. A malignant image therefore has roughly twice the chance of being selected compared with a benign image, making the expected training exposure close to balanced.

Why this is useful - and what it changes

Oversampling helps the loss function see enough positives and can improve malignant recall. Because sampling is with replacement, a given image can appear more than once in an epoch while others may not appear. Training AUC/loss are therefore measured on a sampled distribution; validation and test metrics remain on the original distribution.

4. Model architecture and two-stage transfer learning

The base model is torchvision's ResNet50 with default ImageNet weights. ResNet50 is a convolutional residual network: residual connections allow information and gradients to pass through deep layers effectively. Its existing visual features - edges, textures, shapes, and progressively more abstract patterns - provide a strong starting point when the medical dataset is much smaller than ImageNet.

Classifier head

The original 1,000-class ImageNet classifier is replaced. ResNet50's pooled feature vector has 2,048 values. The new head maps 2,048 -> 512 -> 128 -> 1 with ReLU activations and dropout 0.23 after each hidden layer. The one final value is a **logit**, not a probability; sigmoid later maps it to a value from 0 to 1.

Layer	Role	Why it is present
Linear 2048 -> 512	First task-specific projection	Adapts general ResNet features to a compact lesion-classification representation.
ReLU	Non-linearity	Lets the head learn more than a single linear boundary.
Dropout 0.23	Regularization	Randomly disables part of the hidden activation during training to discourage dependence on specific units.
Linear 512 -> 128 + ReLU + Dropout	Further compression and regularization	Builds higher-level task features while limiting overfitting.
Linear 128 -> 1	Binary output	Produces one logit for malignant probability.

Stage 1: train only the head

All original ResNet parameters are frozen and only model.fc is optimized for 5 epochs using AdamW with learning rate 0.001. The output reports 24,622,913 total parameters but only 1,114,881 trainable parameters. This gentler first stage rapidly fits a classifier head without immediately changing useful pre-trained visual representations.

Stage 2: unfreeze higher layers

The notebook reloads the Stage 1 checkpoint, freezes all parameters again, then unfreezes layer3, layer4, and the new classifier head. These later ResNet blocks contain more task-specific visual concepts than early layers. Fine-tuning them at the smaller learning rate 0.0003 allows the model to adapt to dermoscopy while reducing the risk of destructive large updates to pre-trained features.

Why two stages are useful

This strategy is a practical compromise: Stage 1 learns a safe task-specific output head; Stage 2 supplies enough flexibility to improve domain fit. The validation AUC rises from 0.8604 at the end of Stage 1 to 0.9418 by the final Stage 2 epoch.

5. Optimization, validation, and checkpoints

The loss, optimizer, scheduler, training loop, and validation loop all work together to update the model while preserving an unbiased view of performance on held-out validation data.

Component	Notebook setting	Why it is useful
BCEWithLogitsLoss	Binary cross-entropy on raw logits	Numerically stable binary loss. It combines sigmoid and cross-entropy internally, so the training code should not apply sigmoid before loss.
AdamW	Head only in Stage 1; unfrozen params in Stage 2	Adaptive optimizer with decoupled weight decay. Weight decay $1e-4$ regularizes weights.
ReduceLROnPlateau	factor 0.22, patience 2	Reduces learning rate after the monitored metric stops improving, allowing smaller corrective steps.
GradScaler + autocast	Enabled when CUDA is available	Implements stable mixed-precision backward passes.
Early stopping	patience 5	Stops if validation loss fails to improve for five epochs and saves a plateau checkpoint.
Checkpointing	<code>torch.save(model, path)</code>	Preserves a model snapshot for later loading. The code saves the full Python model object rather than only its state dictionary.

Training and validation functions

- **train_one_epoch** sets `model.train()`, transfers batches to the selected device, clears old gradients, computes logits and BCE loss under AMP, backpropagates through GradScaler, and updates the optimizer. It collects sigmoid probabilities and computes ROC-AUC across the entire epoch.
- **validate** sets `model.eval()` and uses `torch.inference_mode()`. This disables gradient tracking, saving memory and compute. The validation loop otherwise computes loss and AUC in the same way, so training and validation metrics are comparable.
- The explicit `.squeeze(1)` changes model output shape from `[batch, 1]` to `[batch]`, matching the label tensor shape `[batch]` and avoiding unintended broadcasting.

Recorded training result

Stage	Best validation loss	Best validation AUC observed	Notes
Stage 1 (5 epochs)	0.4400 at epoch 3	0.8604 at epoch 5	Head-only learning; scheduler output shows LR $2.20e-4$ after plateau.
Stage 2 (12 epochs)	0.3775 at epoch 8	0.9418 at epoch 12	Layer3/layer4/head fine-tuning; LR reaches $1.45e-5$.

Important implementation observation

Stage 1 creates `ReduceLROnPlateau` with `mode='max'` but passes `val_loss` into `scheduler.step()`. Because lower loss is better, this monitor should use `mode='min'`, as Stage 2 correctly does. The Stage 1 scheduler may therefore reduce the learning rate based on the wrong improvement direction. This is a code-quality issue, not a reason to discard the saved test result, but it should be corrected before rerunning.

6. Final test evaluation and interpretation

After training, the notebook loads a specified fine-tuned checkpoint, runs every test image once in evaluation mode, turns logits into probabilities with sigmoid, and classifies a case as malignant when probability ≥ 0.50 . It computes ranking metrics from probabilities and threshold-dependent metrics from the binary decisions.

0.8421	accuracy
0.9153	ROC-AUC
0.7575	malignant precision
0.7766	malignant recall
0.8750	specificity
0.7669	malignant F1

What the metrics mean

Metric	Result	Plain-language interpretation
ROC-AUC	0.9153	The model usually ranks a randomly chosen malignant image above a randomly chosen benign image. It summarizes discrimination across possible thresholds.
Accuracy	84.21%	3,200 of 3,800 test images were classified correctly at threshold 0.50.
Precision	75.75%	Among images predicted malignant, about three in four were malignant in this test set.
Recall / sensitivity	77.66%	The model detected 987 of 1,271 malignant-labelled test images; 284 were missed at the chosen threshold.
Specificity	87.50%	The model correctly identified 2,213 of 2,529 benign-labelled test images; 316 were false alarms.
F1 score	0.7669	Harmonic balance of malignant precision and recall at the 0.50 threshold.

Confusion matrix reconstructed from reported metrics

True class / predicted class	Predicted benign	Predicted malignant	Row total
Benign	2,213 (87.50%)	316 (12.50%)	2,529
Malignant	284 (22.34%)	987 (77.66%)	1,271
Column total	2,497	1,303	3,800

Clinical-use caution

These are research-model results, not evidence that the system is ready to diagnose patients. A deployment decision would require external validation, patient-level leakage checks, calibration and threshold selection for the intended clinical setting, fairness analysis, and expert clinical governance.

7. Visual reporting in the notebook

The notebook combines Stage 1 and Stage 2 history arrays and plots three figures. A vertical dashed line denotes the start of Stage 2. These visualizations are useful because numeric epochs alone make it hard to see convergence, divergence, and learning-rate changes.

Figure	What it plots	How to read it
Complete training loss	Train loss and validation loss by epoch	Training loss falling while validation loss rises can signal overfitting. The best recorded validation loss is 0.3775 at Stage 2 epoch 8.
Complete training AUC	Train AUC and validation AUC by epoch	Shows ranking quality. Training AUC rises to 0.9970 while validation AUC ends at 0.9418; the gap is a cue to monitor overfitting.
Learning rate during training	Learning rate on a logarithmic y-axis	Shows when ReduceLROnPlateau stepped down the optimizer learning rate, especially during Stage 2.
Confusion matrix (count and percent)	Errors by true/predicted class	Shows the type and number of errors, which aggregate accuracy alone hides.

Why ROC-AUC and loss are both tracked

Loss measures how well predicted probabilities match labels and is sensitive to confidence. ROC-AUC measures only whether positives tend to receive higher scores than negatives. A model can improve AUC without improving calibrated probabilities, or vice versa, so monitoring both gives a richer training picture. The code chooses best checkpoints by validation loss, not AUC.

Why the classification threshold matters

The threshold of 0.50 is simple and conventional, but it is a modelling choice. Lowering it would likely increase malignant recall while creating more benign false positives; raising it would have the opposite effect. A healthcare screening system would normally select its threshold from validation data using a predefined safety objective rather than assume 0.50 is optimal.

Notebook diagnostic / speed-test cells

Cells 41-47 are not part of model learning. They verify tensor shapes, data loading time, GPU transfer, memory usage, and a complete batch update before the full run. These checks are highly useful for finding bottlenecks and configuration errors early. Their saved outputs show a batch size of 64, a peak GPU use of 0.70 GB in one test, and successful CUDA transfers.

8. Reproducibility and recommended refinements

The notebook is well structured as an experimental pipeline, but the following refinements would make a rerun more reliable and its claims easier to interpret.

Priority	Observation	Recommended action
High	RANDOM_SEED = 42 is declared but torch, NumPy, and Python random seeds are not set in saved cells.	Call random.seed, np.random.seed, torch.manual_seed, torch.cuda.manual_seed_all; document deterministic/cudnn trade-offs.
High	Stage 1 scheduler uses mode='max' while receiving validation loss.	Change to mode='min', matching Stage 2, or monitor validation AUC with mode='max'.
High	The final loading cell names a plateau checkpoint at epoch 10, while the loop says the best loss checkpoint is at epoch 8.	Explicitly record why that path was selected; normally evaluate the best validation checkpoint consistently.

Priority	Observation	Recommended action
High	The split is image-level; the notebook does not show patient or lesion identifiers.	Where metadata permits, perform patient-level splitting to avoid correlated images appearing across splits.
Medium	Full model objects are saved with <code>torch.save(model)</code> .	Prefer saving <code>state_dict</code> plus architecture/config metadata; it is safer and more portable across code versions.
Medium	A fixed 0.50 threshold is used without calibration/selection.	Choose a threshold on validation data and report sensitivity-specificity trade-offs, PR-AUC, and calibration.
Low	<code>PREFETCH_FACTOR</code> is set but not passed to <code>DataLoader</code> ; <code>cv2</code> is installed but unused.	Remove unused settings/imports or use them deliberately, so the notebook is easier to audit.

Appendix A. Cell-by-cell map

This map relates the notebook's executed cells to the conceptual workflow. Empty markdown headings and simple print-only cells are grouped with the nearest functional stage.

Cells	Role	Why it exists
1-3	Install/import packages; select device	Sets up dependencies and chooses CUDA when present.
5-6	Configuration and GPU optimizations	Centralizes experimental hyperparameters and acceleration settings.
9-12	Read CSV and create binary label	Inspects the labels, selects MEL/BCC/SCC, and counts classes.
14	Verify image paths	Ensures every CSV row maps to a usable image file.
16-17	Split and transform data	Creates stratified sets and defines train vs evaluation preprocessing.
20-24	Dataset, sampler, loaders	Feeds transformed image-label batches efficiently and balances training exposure.
27-33	Model, optional compilation, loss/optimizer	Creates the ResNet50 head, reports trainable parameter count, and prepares Stage 1 learning.
36-38	Training/validation functions and checkpoint folder	Encapsulates repeated epoch work and creates storage for snapshots.
41-47	Batch/GPU diagnostic checks	Tests data path and GPU throughput before long training.
49-50	Stage 1 training and reload	Trains the classifier head, tracks validation, saves/reloads checkpoint.
52-56	Stage 2 setup and fine-tuning	Unfreezes deeper layers, lowers LR, trains, and loads selected fine-tuned checkpoint.
58	Combined learning curves	Compares both stages across loss, AUC, and learning rate.
59	Final test metrics	Generates accuracy, AUC, precision, recall, F1, specificity, reports, and matrices.

Appendix B. Key configuration values

Parameter	Value	Use in code
IMAGE_SIZE	224	Resizes all inputs to ResNet50's expected spatial size.
BATCH_SIZE	64	Number of samples processed per batch.
FROZEN_EPOCHS / FINETUNE_EPOCHS	5 / 12	Maximum epochs for the two training stages.
FROZEN_LR / FINETUNE_LR	0.001 / 0.0003	Higher initial head-learning rate; smaller fine-tuning rate.
DROPOUT	0.23	Regularizes the custom classifier head.
WEIGHT_DECAY	1e-4	AdamW regularization strength.
EARLY_STOPPING_PATIENCE	5	Validation-loss non-improvement tolerance before stop.
LR_SCHEDULER_PATIENCE / FACTOR	2 / 0.22	Controls LR decrease on a plateau.
USE_AMP / USE_TORCH_COMPILE	True / False	AMP participated in CUDA run; compilation did not.

Conclusion

The notebook is a complete, performance-conscious transfer-learning experiment with a sensible data workflow and a strong recorded ranking result. Its most important next steps are to correct the Stage 1 scheduler direction, make reproducibility explicit, confirm the intended checkpoint, and validate beyond this single held-out split before drawing clinical conclusions.